

A PROJECT REPORT ON

Raksha: Enhancing Women's Safety through AI-based Violence Detection

**SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE IN THE PARTIAL FULFILLMENT FOR THE AWARD OF
THE DEGREE**

**OF
BACHELOR OF ENGINEERING
IN
INFORMATION TECHNOLOGY
BY**

Rohan Punde	B400070545
Tushar Jadhav	B400070509
Mayur Suryavanshi	B4000705026
Vivek Pawar	B400070539

**UNDER THE GUIDANCE OF
PROF. S. V. Aslekar**



DEPARTMENT OF INFORMATION TECHNOLOGY

**PUNE VIDYARTHI GRIHA'S COLLEGE OF ENGINEERING AND
TECHNOLOGY & G K PATE(WAND) INSTITUTE OF MANAGEMENT 44,
VIDYANAGARI, PARVATI, PUNE-411009, MAHARASHTRA, INDIA**

2024-25



CERTIFICATE

This is to certify that the project report entitled

Raksha: Enhancing Women's Safety through AI-based Violence Detection

Submitted by

Rohan Punde	B400070545
Tushar Jadhav	B400070509
Mayur Suryavanshi	B400070526
Vivek Pawar	B400070539

is a bonafide work carried out by them under the supervision of Prof.S.V. Aslekar and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University for the award of the Degree of Bachelor of Engineering (Information Technology)

Prof.S.V. Aslekar
Internal Guide
Department of IT

Dr.S.A.Mahajan
Head of Department
Department of IT

External Examiner
Date :
Place : Pune

Dr. M.R. Tarambale
Principal
PVGCOET & GKPIOM, Pune

ACKNOWLEDGEMENT

We are delighted to submit this report on “**Raksha: Enhancing Women’s Safety through AI based Violence detection.**”

First and foremost, we owe sincere gratitude to our project guide, **Prof. S. V. Aslekar** , whose seasoned guidance, continual encouragement, and insightful feedback steered every phase of this work. Their expertise proved invaluable in refining our ideas and elevating the overall quality of the project.

Our heartfelt thanks also go to **Dr. S. A. Mahajan**, Head of the Information Technology Department at PVGCOET & GKPIOM, Pune. The resources, facilities, and motivating environment he provided enabled us to carry out this project smoothly. His confidence in our abilities gave us the impetus to persevere and excel.

We further acknowledge the unwavering support of the entire Information Technology Department staff, who offered practical assistance and timely advice whenever needed.

Finally, we extend deep appreciation to our families and friends for their constant encouragement, patience, and belief in us throughout this journey. Their support formed the backbone of our efforts and success.

Rohan Punde

Tushar Jadhav

Mayur Suryavanshi

Vivek Pawar

(B.E Information Technology)

List of Figures

Fig.No	Diagram Name	Page No
7.1	Architecture Diagram	20
8.1	Use Case Diagram	23
8.2	Sequence Diagram	24
8.3	Activity Diagram	25
8.4.1	DFD L-0 Diagram	26
8.4.2	DFD L-1 Diagram	26
8.5	Class Diagram	27

CONTENTS

Sr no		Contents	Page no
1		Introduction	1
	1.1	Aim	2
	1.2	Motivation	2
	1.3	Objectives	2
2		Literature Survey	3
3		Problem Definition	9
4		Software Requirement Specification	10
	4.1	Functional Requirements	12
	4.2	Non Functional Requirements	13
	4.3	System Requirements	14
5		Project Requirement Specification	15
6		Flowchart	19
7		Architecture	20
8		UML Diagram	23
	8.1	Use Case Diagram	23
	8.2	Activity Diagram	24
	8.3	Sequence Diagram	25
	8.4	Data Flow Diagram (DFD 0 1 2)	26
	8.5	Class Diagram	27
9		Implementation	28
10		Result and Evaluation	33
11		GUI	37
12		Project Plan	44
13		Conclusion	48
14		References	49
15		Annuxure A	50

ABSTRACT

This project seeks to develop an AI-driven system focused on detecting violent behaviors to improve the safety of women in both public and private environments. Leveraging machine learning (ML) techniques, the system will be designed to analyze audio and visual inputs from readily available sensors such as cameras and microphones. The primary goal is to recognize signs of aggression, distress, or abnormal behaviors such as loud voices, sudden movements, or altercations that may indicate potential threats or violence.

The system will operate in real-time, continuously monitoring its surroundings for any indication of violent actions or behavior that could put individuals at risk. Upon detecting such behavior, the system will generate alerts and promptly notify local authorities or pre-identified emergency contacts. This immediate response mechanism is intended to facilitate quick intervention, potentially preventing further escalation and enhancing the protection of women in vulnerable situations. Developed within resource constraints, the project aims to demonstrate the practicality of applying artificial intelligence to real-world problems like violence detection. The research and development efforts will provide valuable insights into the feasibility of deploying AI for real-time violence detection and response, with potential applications extending beyond women's safety to broader societal use cases.

1 Introduction

The growing concern over the safety of women in today's society has highlighted the urgent need for intelligent systems capable of detecting and responding to violent or suspicious behavior in real time. Traditional surveillance systems, though helpful for monitoring, rely heavily on manual observation and are limited in their ability to interpret aggressive actions or intervene promptly. This limitation creates a critical demand for automated, AI-driven surveillance systems that can operate autonomously and respond instantly to potential threats.

The proposed project, titled **“Raksha: Enhancing Women's Safety through AI-based Violence Detection,”** aims to bridge this gap by developing an intelligent system that analyzes both audio and visual inputs to identify signs of violence, aggression, or distress. By utilizing readily available hardware such as CCTV cameras and microphones, the system can monitor an environment continuously and interpret behaviors like sudden movements, physical altercations, or loud voices that often precede acts of violence.

Powered by machine learning algorithms and deep learning models, the system is capable of real-time detection and alert generation. Once any abnormal activity is recognized, it will immediately notify local authorities or predefined emergency contacts, ensuring timely intervention and potentially preventing harm. This automated alert mechanism is especially vital in high-risk scenarios where human response may be delayed or unavailable.

Designed with cost-effectiveness and scalability in mind, the system is intended to function within limited resources, making it accessible for various settings, including residential areas, public spaces, educational institutions, and workplaces. The project is also adaptable to existing surveillance infrastructure, reducing the need for additional equipment or major upgrades.

While the primary focus of this system is women's safety, its applications extend to broader use cases such as school safety, workplace conflict monitoring, and public space surveillance. Additionally, the insights gained during this project's research and development phase will contribute to the growing field of AI-based behavioral analysis and enhance future efforts in real-time public safety technologies.

By combining AI technologies with real-time surveillance, this project aspires to create a proactive safety solution that transforms passive monitoring systems into active protectors. It also contributes to the evolving field of intelligent surveillance and offers insights into the feasibility and impact of AI-based violence detection in real-world environments.

1.1 Aim :

The primary aim of this project is to design and develop an intelligent surveillance system

capable of detecting suspicious or violent behavior in real time, using deep learning techniques.

The specific aims include:

- **To build an AI-based system** that can analyze live audio and video feeds from cameras and microphones to identify aggressive, violent, or distressful behavior.
- **To ensure real-time monitoring and response** by developing a model that triggers instant alerts upon detecting potential threats, enabling timely intervention.
- **To enhance women's safety** in public and private spaces by reducing human dependency in surveillance and enabling automated threat detection.
- **To create a cost-effective and scalable solution** that can be implemented using existing infrastructure, making it practical for widespread adoption.

1.2 Objective :

- **To analyze real-time audio and video inputs** using deep learning models to identify signs of aggression, violence, or distress.
- **To develop a responsive alert mechanism** that notifies local authorities or emergency contacts upon detecting suspicious behavior.
- **To design a cost-effective and scalable system** that can be deployed using existing surveillance infrastructure with minimal upgrades.
- **To enhance safety and security for women** by enabling proactive monitoring in both public and private environments. To provide real-time predictions and alert mechanisms.

2 Literature Survey

2.1 Introduction

With the rise in safety concerns, especially related to women in public and private spaces, the development of intelligent surveillance systems has gained significant attention. Traditional surveillance relies heavily on manual monitoring, which is often inefficient, error-prone, and delayed in response. Recent advancements in Artificial Intelligence (AI), particularly in Machine Learning (ML) and Deep Learning (DL), have enabled the automation of surveillance through Human Activity Recognition (HAR) and anomaly detection. These technologies allow systems to detect violent or suspicious behavior by analyzing audio and visual cues such as sudden movements, loud voices, and physical altercations. Various models, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), MobileNet, and Bi-GRU, have shown promising results in real-time behavior analysis. Research in this domain focuses on improving detection accuracy, minimizing false positives, and ensuring real-time alerts. This literature review explores the existing work, tools, and techniques that form the basis of building an effective AI-powered surveillance system aimed at enhancing safety and enabling timely intervention in critical situations.

2.2 Background and Framework

Human Activity Recognition (HAR) refers to the automated detection and classification of human actions or behaviors through sensor data or visual inputs, most commonly video feeds. In surveillance applications, HAR plays a crucial role in identifying unusual or potentially dangerous activities such as physical altercations, theft, or the display of weapons. Earlier approaches to HAR primarily relied on manually crafted features derived from techniques like background subtraction, optical flow, or motion trajectory analysis. However, these traditional methods often fall short in complex, real-world scenarios involving occlusions, varying lighting conditions, or multiple individuals interacting simultaneously.

The introduction of deep learning has transformed HAR by enabling more robust and accurate behavior recognition. Convolutional Neural Networks (CNNs), such as MobileNetV2, excel at extracting spatial features from video frames, while Recurrent Neural Networks (RNNs), including Bidirectional Gated Recurrent Units (Bi-GRU), effectively capture temporal relationships between consecutive frames. Together, these models enhance the system's ability to recognize and interpret human activity in dynamic environments.

2.3 Review of Existing Solutions

Year	Name	Survey Topic Summary	Limitation
2024	Zaidi et al.	Proposed a multi-stream approach using CNN, ConvLSTM, and GRU to classify activities like snatching, kicking, and shooting using a custom dataset of 706 videos.	High computational requirements due to complex temporal models.
2023	Ullah et al.	Used a Time-Distributed CNN with Conv3D; achieved over 90% accuracy and proved viable for real-time deployments.	High computational cost; less suited for low-resource environments.
2024	Namithadevi et al	Integrated MobileNetV2 with LSTM to reduce model size while maintaining accuracy, ideal for low-power or edge deployments.	Slight compromise in accuracy in highly complex scenes.
2023	Sampedro et al.	Applied Bi-GRU to Human Activity Recognition tasks, showing that bidirectional learning improved sequential understanding of actions in videos.	Focus was more on HAR; limited evaluation in diverse surveillance use cases
2022	Banerjee et al.	Introduced explainable AI frameworks using attention and post-hoc tools for model interpretability in surveillance applications.	Adds complexity; may impact real-time performance.
2021	Xu et al	Used a two-stream fusion (RGB + optical flow) approach to better detect abnormal behavior.	Complex architecture; requires dual input processing

			and tuning.
2023	Kumar et al	Employed standard CNNs for classifying suspicious activities; achieved promising results.	Lacked temporal modeling, reducing effectiveness for sequential tasks.
2023	Shinde et al	Used CNN-LSTM architecture for high-level activity recognition; delivered accurate classification.	Requires GPU; not suitable for low-power or real-time edge deployment.
2023	Gupta et al	Emphasized preprocessing and video enhancement to improve detection performance, especially in poor lighting.	Did not integrate enhancement techniques directly into deep learning pipeline.
2018	Wang et al.	Applied RNNs to classify behavior sequences in video footage; effective in understanding temporal behavior	Designed for static settings; lacked optimization for realtime or dynamic environments.

2.4 Distinguishing the Proposed Work

- Existing approaches to suspicious activity detection often rely on complex deep learning models like CNN-LSTM or ConvLSTM, which, while accurate, are computationally expensive and unsuitable for real-time or edge-based surveillance systems.
- Our proposed solution addresses this gap by integrating two lightweight yet effective models:
 - **MobileNetV2:**
 - Designed for efficient spatial feature extraction with significantly fewer parameters.
 - Ideal for real-time performance in low-resource environments such as smart cameras and embedded systems.
 - Reduces memory consumption and processing time, making it compatible with edge deployment.
 - **Bi-GRU (Bidirectional Gated Recurrent Unit):**
 - Offers efficient temporal analysis by learning from both past and future frames.
 - Less computationally heavy compared to LSTM and ConvLSTM while maintaining comparable accuracy.
 - Well-suited for capturing contextual information in sequential video data such as ongoing physical confrontations.
- A key innovation is the introduction of our custom dataset:
 - Specifically designed to reflect real-world CCTV surveillance environments with fixed-angle views.
 - Contains a wide range of realistic scenarios, including physical violence, snatching, and visible weapons.
 - Addresses limitations of public datasets that use mobile footage, cinematic shots, or uncontrolled camera angles.

2.5 Methodologies and Research Process

- The research was conducted with a clear focus on **real-time, deployable solutions** for public surveillance systems in smart cities.
- A thorough **literature review** was performed, covering:
 - Over **ten peer-reviewed articles** on HAR (Human Activity Recognition), violence detection, and surveillance technologies.
 - Emerging trends like hybrid architectures (e.g., CNN-LSTM, Conv3D-GRU), lightweight networks, and the importance of explainability in AI.
- Multiple model architectures were developed and evaluated:
 - **CNN-LSTM**: Effective but heavy on GPU resources, not ideal for edge use.
 - **Time Distributed CNN + GRU**: Good temporal modeling but slower inference.
 - **CNN-BiGRU**: Offered the best trade-off between speed and accuracy, selected as the final model.
- Extensive **benchmarking** was carried out based on:
 - **Model accuracy** on the SCVD dataset.
 - **Training time** on GPU platforms (Google Colab with Tesla K80/T4).
 - **Inference latency**, especially relevant for real-time use cases.
- The final model demonstrated superior performance, making it the foundation for our smart surveillance system.

2.6 Tools, Technologies and Resources Used

A combination of modern tools and platforms were utilized to ensure an efficient, scalable, and real-time deployable solution:

- **Programming & Modeling:**
 - Python: Used for data preprocessing, training pipelines, and integration.
 - TensorFlow & Keras: Chosen for their high-level APIs and support for edge deployments using TensorFlow Lite.
- **Computer Vision & Data Handling:**
 - OpenCV: Facilitated real-time frame capture and preprocessing for both training and inference.
- **Training Infrastructure:**
 - Google Colab: Provided access to cloud-based GPUs for model training and experimentation.
- **Annotation Tools:**
 - LabelImg & CVAT: Used for frame-by-frame annotation of violent, normal, and weapon-related activities.
- **Custom Dataset:**
 - **SCVD Dataset:** Curated by collecting real CCTV footage under various lighting, angle, and crowd density conditions.
 - Labeled with three classes: **Normal**, **Violence**, and **Weapon** to ensure robust multiclass classification.
- **Model Selection Justification:**
 - **MobileNetV2:** Chosen for its mobile-first architecture, allowing deployment on edge devices such as Raspberry Pi, Jetson Nano, or IP Cameras with built-in AI modules.
 - **Bi-GRU:** Enabled bidirectional sequence learning with less computational burden, enhancing performance in streaming surveillance footage.

3 Problem Definition

In today's world, the safety of individuals—especially women—in public spaces is a growing concern due to the rising instances of violence and harassment. Traditional surveillance systems are often passive and rely heavily on human monitoring, which can lead to delayed response times and oversight. There is a need for an intelligent system that can **automatically detect violent activities or harassment in real-time** and immediately notify authorities to take swift action.

This project aims to develop an **AI-based real-time surveillance system** that utilizes both **video (camera) and audio (microphone) inputs** to detect violent behavior and incidents of harassment. The system will leverage machine learning and deep learning techniques to analyze environmental cues and trigger alerts to nearby security personnel or authorities for immediate response.

The goal is to enhance public safety, especially for women, by enabling **automated, real-time detection and alerting mechanisms**, thereby reducing dependence on manual surveillance and improving the speed and accuracy of emergency responses.

4 Software Requirements Specification

4.1 Functional Requirements

The proposed smart surveillance system uses real-time video and audio analytics integrated with deep learning to detect violent or suspicious activities and alert authorities—specifically focusing on public safety and women protection.

1. Real-Time Video and Audio Input

The surveillance system must support continuous live video feed acquisition from CCTV cameras and audio from camera microphone deployed at strategic locations. The cameras are expected to provide high-resolution footage with minimal latency and audio with Noise suppression and silence trimming. The input module should be robust enough to handle:

- Live feed input from CCTV cameras and microphones at strategic locations. Frame acquisition without lag or dropped frames.
- Handle :
 - Multiple video/audio streams simultaneously.
 - Low-latency frame/audio capture.
 - Adverse conditions: low lighting, noisy environments, crowd density.

This ensures the surveillance infrastructure remains responsive and reliable in dynamic public or private settings like schools, malls, transportation hubs, and corporate offices.

2. Preprocessing

Video preprocessing and audio processing is an essential step to improve frame quality and reduce noise before feature extraction and classification. The following preprocessing functions are required:

- **Video Frames:**
 - Frame Rate: ~30 FPS to ensure temporal accuracy.
 - Techniques: Gaussian blurring, adaptive thresholding.
 - Redundancy filtering: Background subtraction and motion detection to discard idle frames.

- **Audio:**
 - Noise suppression and silence trimming.
 - Spectrogram conversion for violent sound event detection.

3. Activity and Sound-based Frame/Segment Filtering

To enhance computational efficiency and system responsiveness, only frames that demonstrate motion or possible human activity must be forwarded to the classification module. Motion detection algorithms such as frame differencing or optical flow will be implemented to:

- Filter only significant motion/audio clips using:
 - Frame differencing or optical flow (video).
 - Sound event detection (e.g., shouting, screaming, weapon discharge).
- Keyframe/key audio segment extraction for analysis.

4. Feature Extraction Using MobileNet and Audio CNN

Once relevant frames are filtered, they must undergo feature extraction using the MobileNet architecture and CNN. This lightweight convolutional neural network (CNN) is ideal for embedded and real-time applications due to its efficiency. It will be used to:

- Identify spatial features like edges, contours, shapes, and textures.
- Detect important visual cues like human posture, object outlines, and facial expressions.
- Maintain a balance between computational cost and detection accuracy, making it suitable for deployment on devices with limited processing power.
- Apply CNNs on spectrograms to capture patterns indicating distress or aggression for audio.

5. Model Building and Activity Classification

a. Data Preparation for Training

- Annotate video and audio data into categories like Normal, Violent, Weaponized, and Harassment.

- Include diverse samples from different environments and angles.

b. Model Training

- Use Bidirectional Gated Recurrent Unit (Bi-GRU) networks to learn temporal dependencies across sequential video frames and audio segments.
- Train visual and audio models separately and fuse their outputs for higher accuracy.

C. Model Evaluation

Post-training, the model is assessed using key performance metrics:

- **Accuracy** for overall prediction correctness.
- **Precision and Recall** to handle class imbalance.
- **F1 Score** as a harmonic mean for imbalanced classifications. These metrics help determine the model's reliability in practical deployment.

6. Real-time Activity Classification and Prediction

The trained system must process incoming frames in real-time and categorize them into:

- **Normal:** Regular, non-threatening behavior.
- **Violent:** Physical aggression such as punching, kicking, or fighting.
- **Weaponized:** Presence of threatening objects like guns or knives.
- Harassment (verbal or physical)

Classification must occur with low latency to enable prompt responses.

7. Suspicious Activity Detection

The system should specifically detect predefined suspicious actions that pose security risks.

These include:

- Running (sudden or frantic motion)
- Punching and Kicking (close-range aggression)
- Falling (indicative of a fight or injury)
- Snatching (theft-like motion)

- Shooting (weapon discharge detection)
- Screaming or shouting (from audio input)

Detection accuracy must remain consistent across varying environments and occlusions.

8. Alert Generation and Notification to Security Personnel

When a frame is classified as violent or weaponized, the system should generate:

- **Instant Alerts** through visual/audio cues, push notifications, SMS, or emails.
- **Contextual Details** including timestamp, activity label, and associated frame snapshot.
- **Log Reports** for audit trails and post-incident investigation.

This ensures that security teams are immediately informed and can respond in time to prevent escalation.

9. User Interface (UI) and Dashboard

The surveillance system should include a clean, responsive UI/dashboard that:

- Displays live video feeds from all connected CCTV sources.
- Highlights suspicious activities in real-time with bounding boxes or labels.
- Maintains an alert history log with filtering and search capabilities.
- Enables playback and snapshot access of flagged incidents.

4.2 Non-Functional Requirements

Non-functional requirements define how the system performs its functions, focusing on performance, scalability, reliability, security, and user experience. These characteristics ensure that the smart surveillance system operates effectively in real-world deployments across different environments.

• Performance

The surveillance system is expected to maintain high throughput by processing at least **30 frames per second (FPS)**, in alignment with real-time operational demands. Timely alert generation is critical — the system must detect and classify suspicious activity and trigger

notifications within **2 to 5 seconds** from the point of detection. This responsiveness is crucial for scenarios where immediate intervention can prevent harm.

• Scalability

The architecture should be scalable to accommodate multiple CCTV inputs without affecting performance. Whether deployed in a small office or a large smart city environment, the system should allow parallel processing of multiple video streams and scale horizontally by adding additional processing nodes or devices as needed.

• Reliability

The system must be robust and capable of operating **24/7** without interruptions or crashes. Mechanisms for automatic recovery from failures, frame buffering during connectivity loss, and continuous system health monitoring should be incorporated. Reliability is vital for security environments where system downtime could result in undetected incidents.

• Security

To safeguard sensitive surveillance data, the system must include **authentication mechanisms, access control, and encryption protocols** for video feeds and alert logs. Only authorized personnel should be able to access the dashboard, control camera feeds, or review activity logs. Network-level protection such as secure socket layer (SSL) communication must also be supported.

• Usability

The **user interface (UI)** must be intuitive, clean, and easy to navigate for non-technical security staff. The dashboard should support:

- Real-time monitoring of camera feeds.
- Visual indicators and logs for detected activities.
- Easy playback of recorded alerts.

The UI should ensure minimal training is required for personnel to begin using the system effectively.

- **Compatibility**

The system should work seamlessly with most standard CCTV cameras, supporting both IP-based and analog systems (via converters if necessary). On the software side, it must be deployable on Windows and Linux-based platforms, offering flexibility for various deployment environments. Containerized deployment using Docker may be considered to improve cross-platform compatibility.

4.3 System Requirements

4.3.1 Hardware Requirements:

The hardware must be capable of supporting high-resolution video capture, deep learning inference, and real-time processing without bottlenecks.

- **Processor:**

A multi-core processor such as **Intel Core i5/i7 (or AMD Ryzen 5/7)** is essential. These CPUs offer the necessary processing power for decoding video streams, handling real-time preprocessing, and coordinating inference tasks.

- **Memory (RAM):**

A minimum of **8GB RAM** is required. This allows the system to perform simultaneous operations, including video decoding, feature extraction, and temporary storage of frame buffers during peak processing.

- **Storage:**

A hybrid configuration comprising:

- **256GB SSD:** For faster boot times, rapid read/write operations, and to store the ML models and inference outputs.
- **1TB HDD:** For archival storage of processed video/audio clips, logs, and long-term datasets.

- **CCTV Camera/microphone Specifications:**

1. **Resolution:** At least **1920x1080 (1080p Full HD)** to ensure clarity in feature extraction
2. **Frame Rate:** **30 FPS** minimum to support accurate temporal modeling.
3. **Lens Type:** **Wide-angle lens (e.g., 120°)** to cover broader areas with fewer blind spots.
4. **Connectivity:** **Ethernet or Wi-Fi** support for real-time video streaming over LAN/WAN.
5. **Storage:** Built-in local storage (e.g., SD card support) or ability to **stream to a central processing server**.
6. **Microphone:** Directional with noise cancellation.
7. **Audio quality:** 16-bit at 44.1kHz.

- **Graphics Processing Unit (GPU):**

A GPU such as **NVIDIA GTX 1650 or higher** is recommended to accelerate deep learning inference and training. This significantly reduces the time for sequence modeling and supports concurrent video stream processing.

4.3.2 Software Requirements:

- **Programming Language:** Python 3.x is the preferred language for machine learning, providing extensive libraries and frameworks for model development and deployment.
- **Machine Learning Frameworks and Libraries :**
 - **TensorFlow/Keras:** These are powerful deep learning frameworks used to build and train models. TensorFlow is the backend, and Keras is user-friendly API for model development.
 - **OpenCV:** Used for real-time video processing, including frame extraction, motion detection, and image enhancement operations like background subtraction, edge detection, and object tracking.
 - **Librosa or Torchaudio:** Essential for audio analysis. These libraries are used for converting audio signals into spectrograms, detecting sound events, applying

noise reduction, and extracting temporal and spectral features relevant to violence or distress sounds.

5 Flowchart

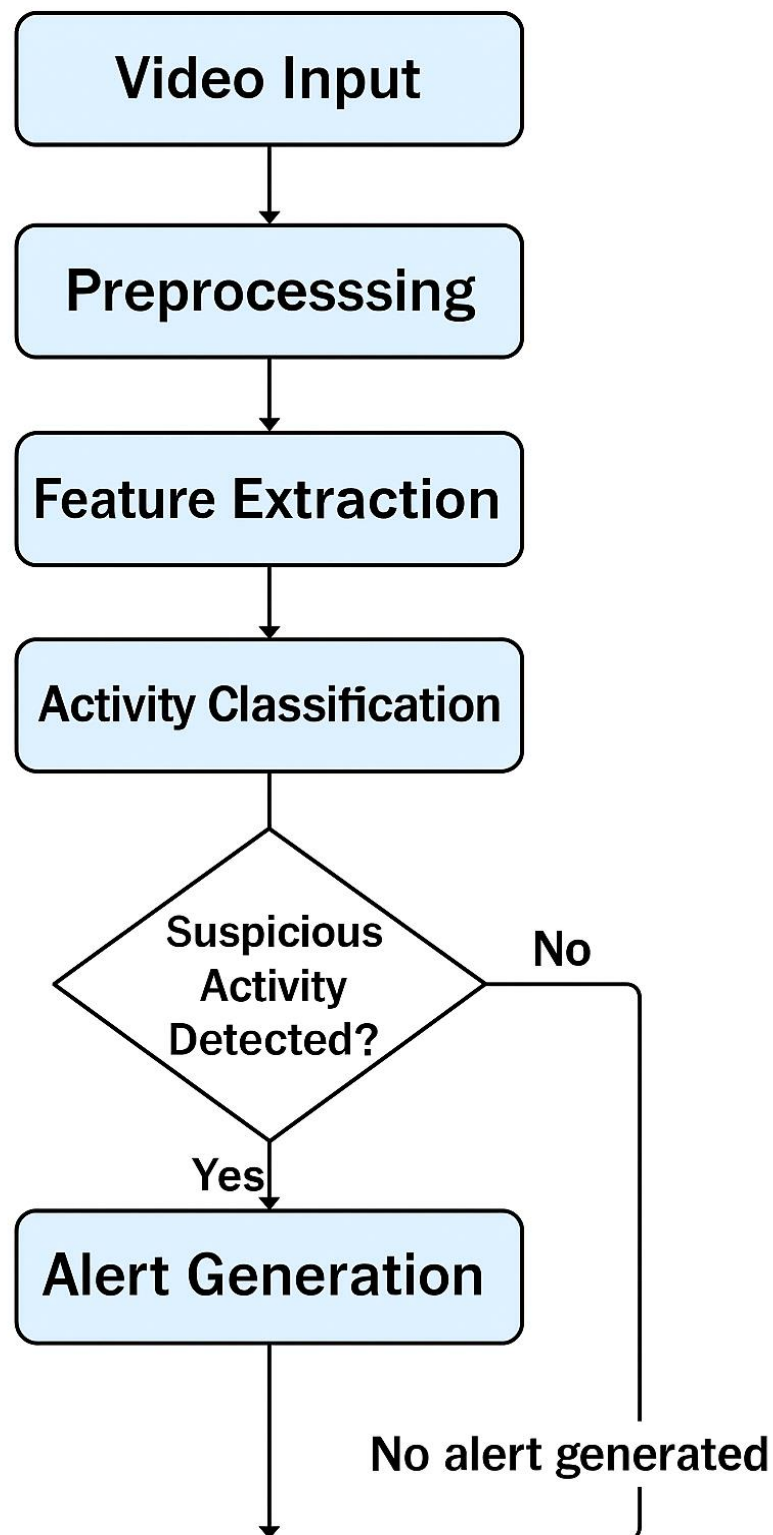


Fig 5.1 Flowchart

6 Project Requirement Specification

6.1 Introduction

The project, titled **Raksha**, aims to build an AI-powered real-time surveillance system focused on women's safety and violence detection. By leveraging live inputs from both the camera and microphone, the system identifies violent behavior or harassment incidents in public spaces (e.g., streets, malls) and immediately informs nearby authorities. Deep learning models such as TensorFlow/Keras, OpenCV, Librosa or TorchAudio are employed to process and classify suspicious activities from video streams.

6.2 Objective

- To create an intelligent surveillance system capable of detecting violence or harassment in real time.
- To classify scenes into Normal, Violent, or Weaponized using deep learning models.
- To send real-time alerts to responsible authorities during critical situations involving women's safety.

6.3 Scope of the Project

- Integration of real-time CCTV and microphone input.
- Preprocessing of video/audio data to reduce noise.
- Frame filtering based on motion/activity.
- Audio analysis to detect aggression (e.g., screaming, loud noises).
- Dashboard for real-time alert notifications.
- System scalability for multiple cameras.

6.4 Functional Requirements

1. Capture and stream video and audio in real time.
2. Preprocess video frames (frame extraction, denoising).
3. Filter frames based on motion.
4. Extract spatial features using MobileNetV2.
5. Apply Bi-GRU to analyze temporal patterns.
6. Classify activity (Normal / Violent / Weaponized).
7. Generate alerts with timestamp and captured frame.
8. Display alerts and video on a user dashboard.

6.5 Non-Functional Requirements

- Performance: Frame processing at ~30 FPS.
- Scalability: Handle multiple cameras simultaneously.
- Reliability: 24/7 continuous processing.
- Usability: Easy-to-use UI for security staff.
- Compatibility: Should support standard CCTV formats and mic hardware.
- Security: Protected access to video data and alerts.

6.6. Hardware Requirements

- 1 Full HD CCTV Camera (1080p, 30 FPS) – ₹7,000
 - External Microphone (Noise-canceling) – ₹3,000
 - Intel i5/i7 CPU (or equivalent) – ₹25,000
 - 8GB RAM – ₹3,000
 - 1TB HDD + 256GB SSD – ₹5,000
 - Optional GPU (if needed later) – Skipped for now
- Total Estimated Hardware Cost: ₹40,000

6.7 Software Requirements

- Python 3.x
- TensorFlow, Keras
- OpenCV
- Librosa / Torchaudio
- Flask/Django for backend
- OpenCV for video processing
- Jupyter Notebook for development
- Ubuntu / Windows 10 OS

6.8 Deliverables

- Trained Deep Learning Model
- Video/audio Stream Integration
- Real-time Suspicious Activity Detection System
- Dashboard + Notification Module
- Project Report and Documentation

7 Architecture

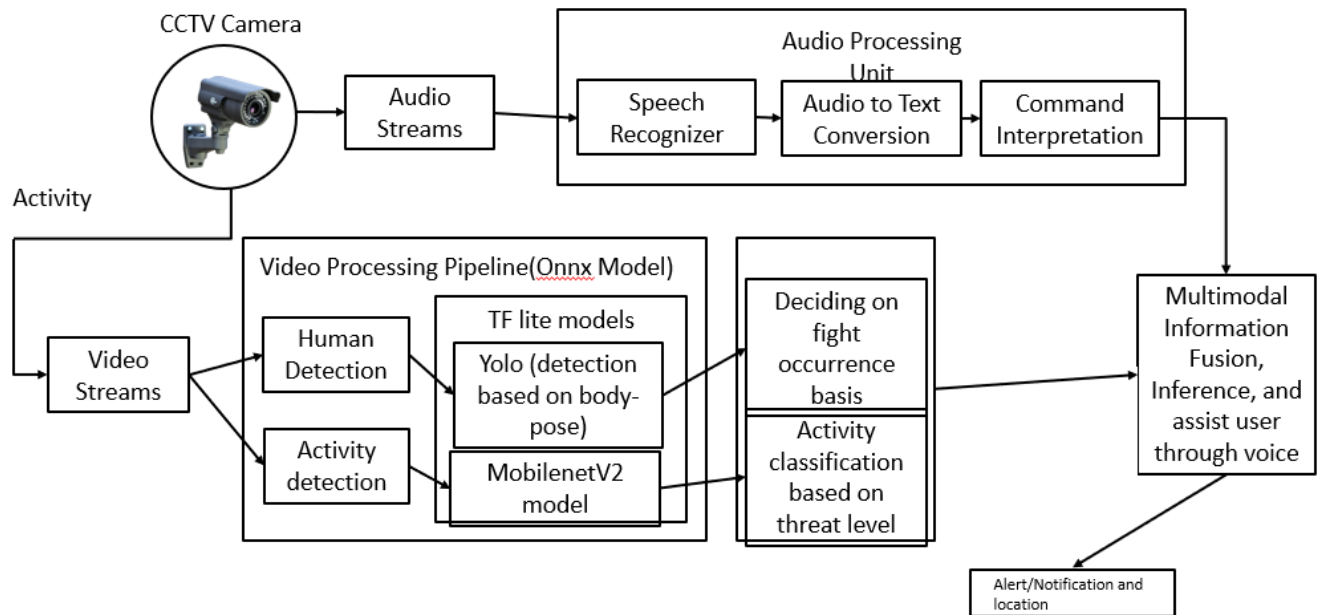


Fig 7.1 : System Architecture

The smart surveillance system follows a structured pipeline consisting of different stages: video and audio input acquisition, preprocessing, feature extraction, model building, prediction, and user interface with alert generation. Each stage contributes to the system's ability to identify suspicious or violent behavior in real time.

• Video and audio Input

The surveillance system begins with real-time video and audio capture from strategically placed **CCTV cameras**. These cameras provide a **continuous stream of high-definition video and audio data**, ensuring comprehensive visual coverage of monitored environments such as public areas, offices, and campuses. The cameras are configured to support live video streaming at a minimum of **30 frames per second (FPS)** to facilitate accurate temporal analysis of activities. Each frame serves as a unit of observation and analysis in the system pipeline.

• Preprocessing

Before model inference, each captured video stream undergoes preprocessing to enhance the quality and relevance of input data. This stage includes:

- **Frame Extraction:** The video is decomposed into individual frames for sequential processing.
- **Noise Reduction:** Techniques such as **Gaussian blurring** and **adaptive thresholding** are used to remove visual noise and background clutter, improving clarity.
- **Normalization and Resizing:** All frames are resized to a consistent dimension (e.g., 224×224 pixels) and pixel values are normalized to suit deep learning model input requirements.
- **Motion Filtering:** Frames with static content or without meaningful activity are discarded to optimize processing and focus on activity-rich segments.

This ensures the incoming data is both clean and informative, reducing the load on subsequent models.

• Feature Extraction

Selected frames from preprocessing are passed through two powerful convolutional neural network (CNN) models — **MobileNet** and **InceptionV3**:

- **MobileNet:** A lightweight CNN optimized for mobile and embedded vision applications. It extracts spatial features such as object outlines, shapes, and foreground elements using depthwise separable convolutions.
- **InceptionV3:** A more complex model capable of learning intricate patterns by employing multiple convolutional filters in parallel. It captures fine-grained and high-level features, such as contextual activity details and spatial configurations.

The **features extracted from both models are concatenated**, resulting in a rich multi-dimensional representation of each frame. This combined representation ensures that both low-level (edges, contours) and high-level (activity context) features are considered.

• Model Building

After feature extraction, the concatenated feature vectors are passed to a **BiGRU (Bidirectional Gated Recurrent Unit)** network for temporal modeling:

- **BiGRU** is a type of recurrent neural network (RNN) capable of learning sequential dependencies in data.
- It processes frame sequences **in both forward and backward directions**, allowing the system to understand activity context from past and future observations.
- The architecture is designed to retain long-term dependencies while reducing vanishing gradient issues, making it ideal for identifying **behavioral patterns** that evolve over time (e.g., movement escalation leading to violence).

This module is trained on labeled datasets containing various human activity sequences categorized as Normal, Violent, or Weaponized.

• **Prediction**

Using the learned temporal and spatial features, the system performs **activity classification**. Each sequence of frames is analyzed and assigned to one of the following classes:

- **Normal:** Regular or harmless human activity.
- **Violent:** Aggressive behavior such as fighting, punching, or pushing.
- **Weaponized:** Actions involving objects such as knives, guns, or blunt tools.

Classification is achieved in real time, with prediction latency kept below **2 seconds** for optimal responsiveness.

• **User Interface and Alert System**

Upon classification of any frame sequence as **violent or weaponized**, the system triggers the alert module. Key functionalities include:

- **Real-Time Notifications:** Security personnel are immediately notified through on-screen alerts, sound beeps, or push notifications (via SMS, email, or mobile app).
- **Incident Logging:** Each alert is logged with the **timestamp, camera ID, predicted activity**, and an **annotated snapshot** of the suspicious frame.

- **Visual Dashboard:** A user-friendly GUI displays live video feeds and highlights detected threats on-screen with bounding boxes or color-coded overlays.
- **Playback and Verification:** Logged incidents can be replayed for verification, facilitating quick response and post-event analysis.

This closed feedback loop ensures the system not only detects but also empowers human operators to act promptly, thus enhancing **public safety and threat mitigation**.

8 UML Diagrams

8.1 Use Case Diagram :

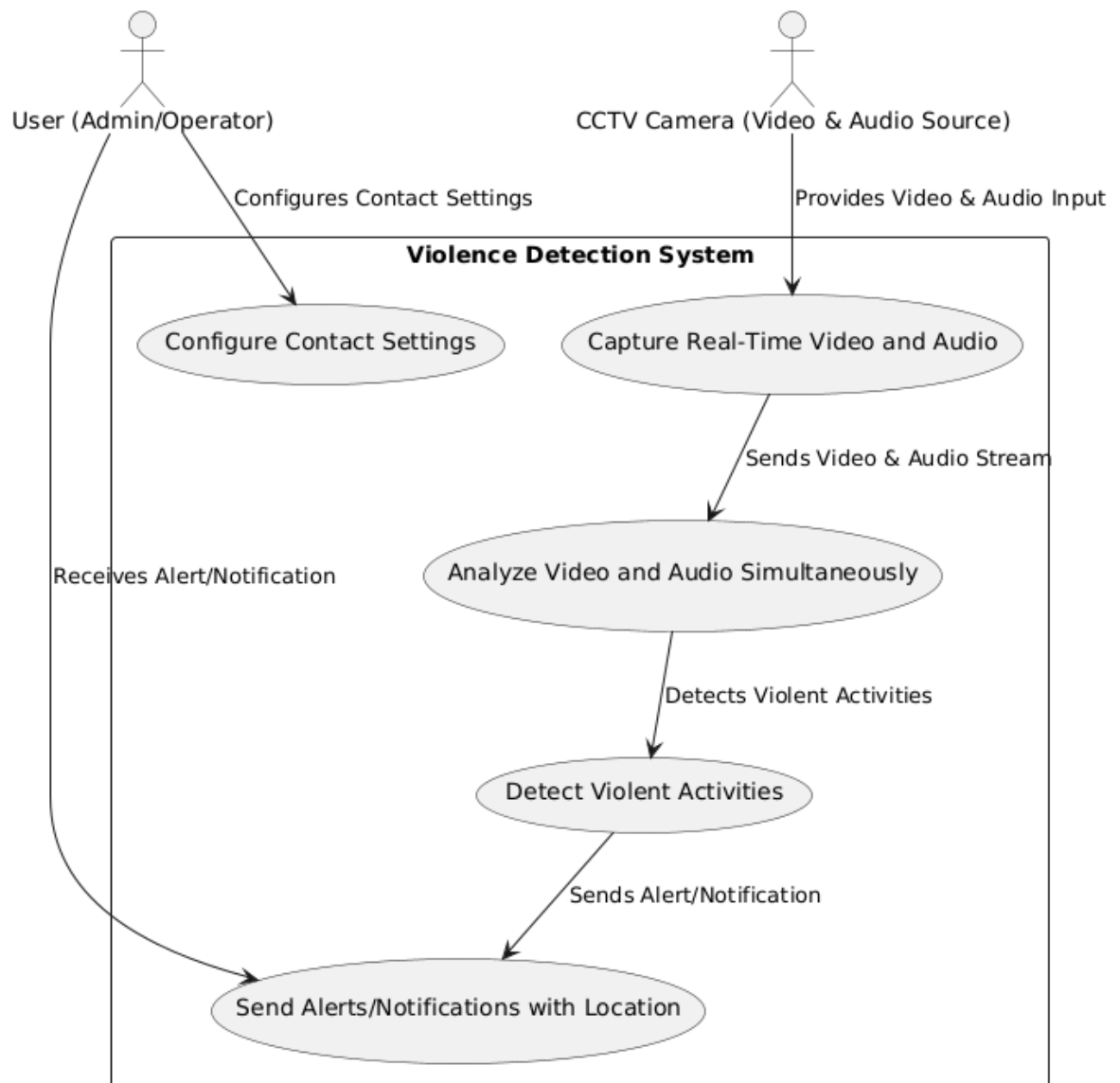


Fig 2 : Use Case Diagram

8.2 Sequence Diagram:

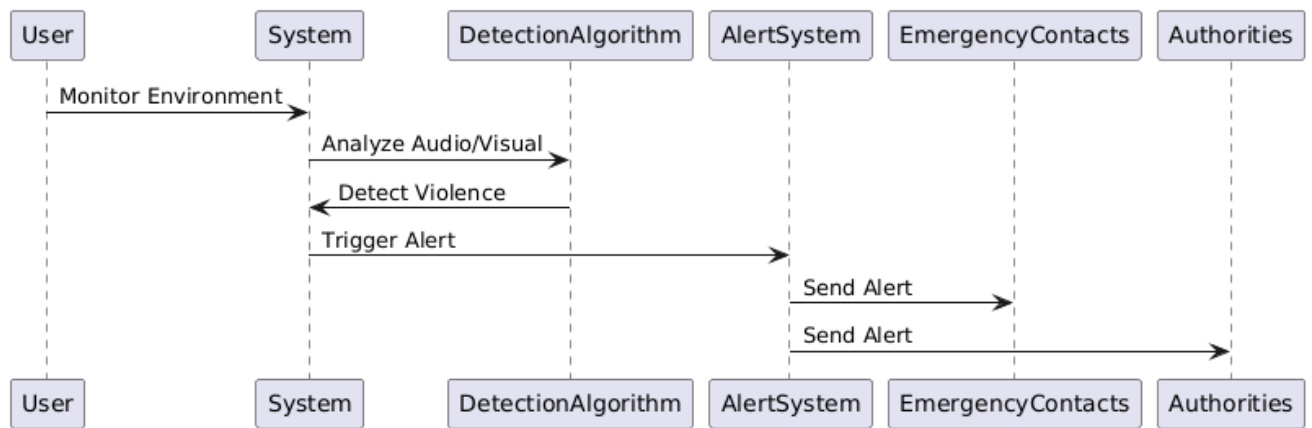


Fig 3 : Sequence Diagram

8.3 Activity Diagram:

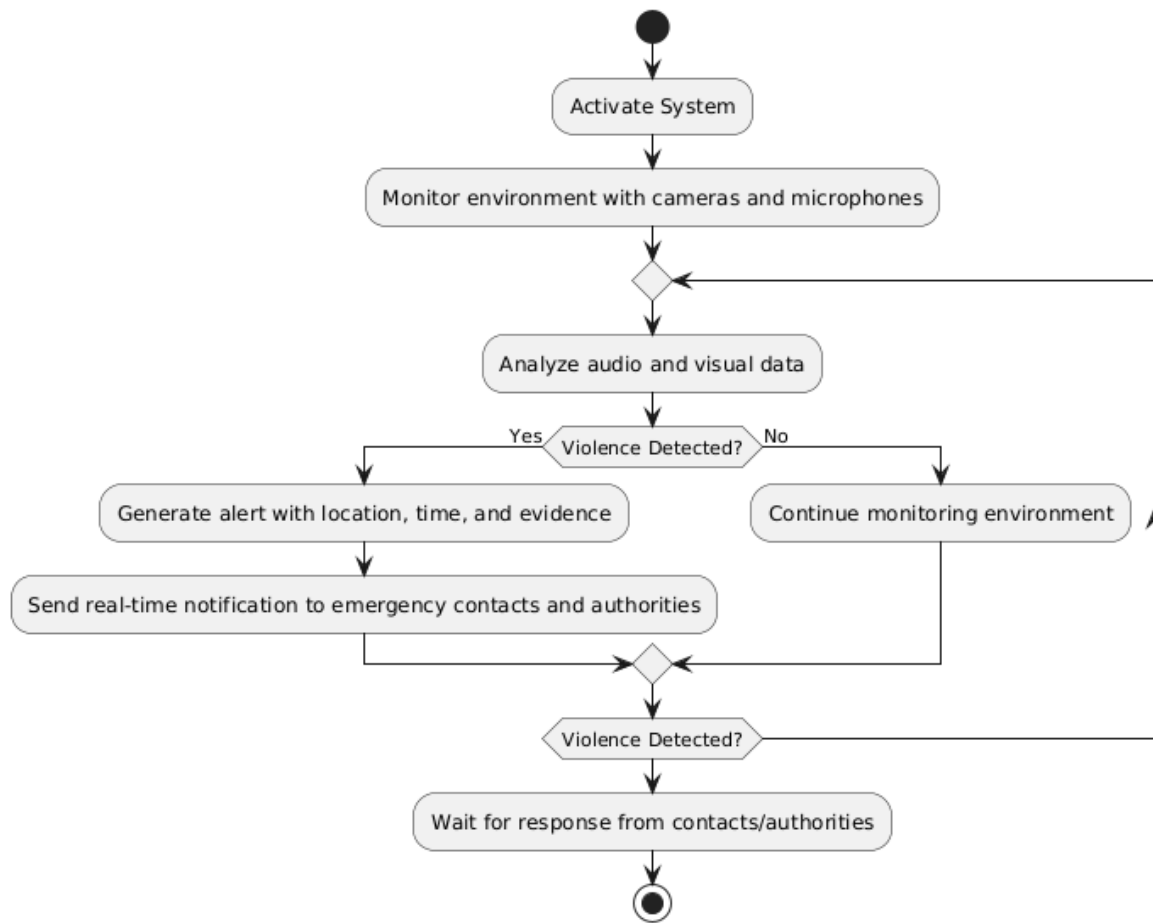


Fig 4 : Activity Diagram

8.4 DFD Diagram:



Fig 5 : DFD level 0

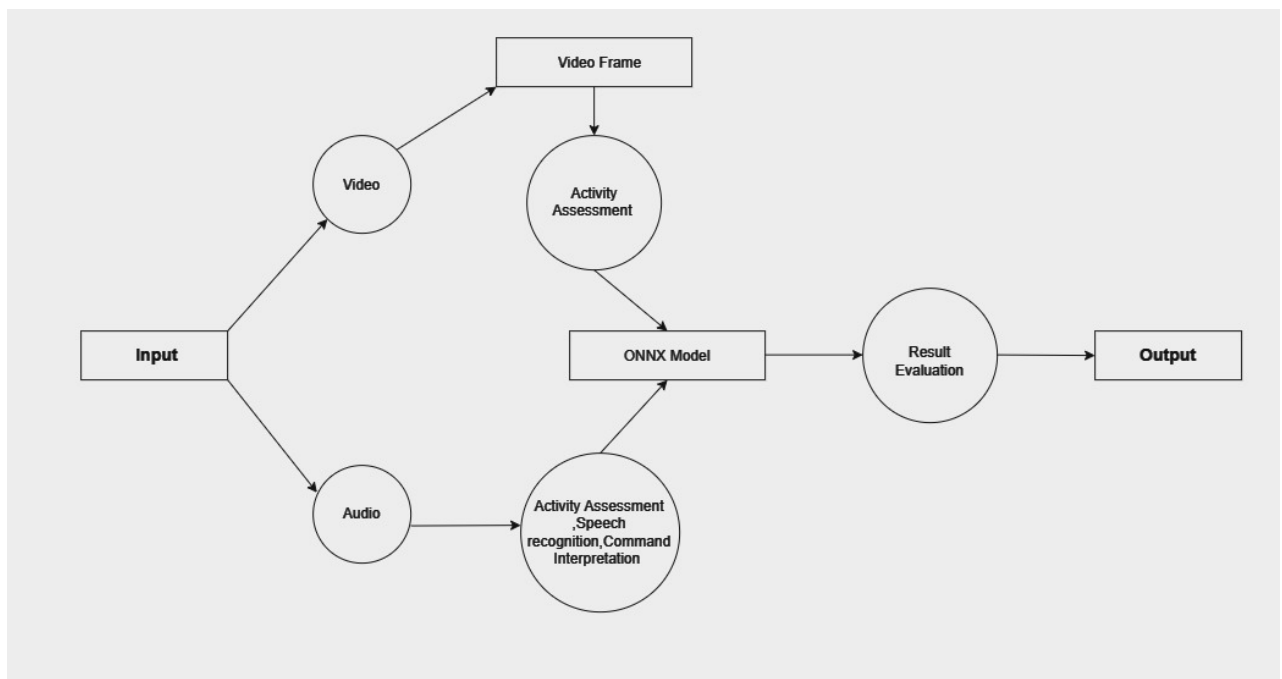


Fig 5 : DFD level 1

8.5 Class Diagram:

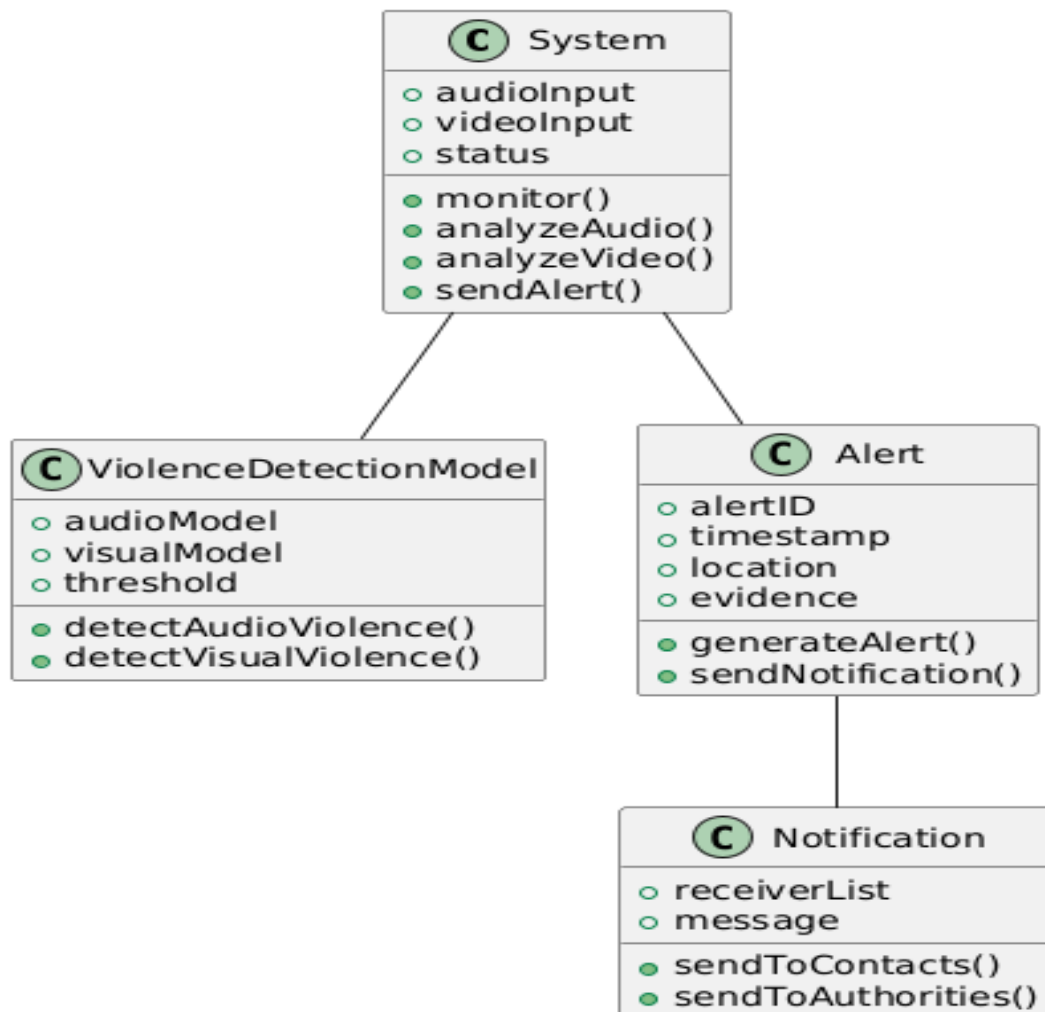


Fig 6 : Class Diagram

9 Implementation

9.1. Data Collection and Preprocessing

The foundation of any deep learning-based surveillance system lies in the quality and relevance of its data. In this phase:

- **Data Sources:** Video data is collected from publicly available surveillance datasets, or captured directly through **camera feeds**. These datasets include different scenarios representing normal human activity, violent actions (such as kicking, hitting), and weaponized behavior (e.g., holding knives or firearms).
- **Frame Extraction:** Continuous video streams are broken down into individual image frames at a rate of **30 frames per second (FPS)**. This conversion is critical for per-frame analysis and reduces the computational complexity of processing entire video clips.
- **Motion Filtering:** To ensure only relevant data is used, **motion detection algorithms** (e.g., frame differencing or background subtraction) are applied to filter out frames without activity. Only frames exceeding a predefined **activity threshold** proceed for feature extraction.

9.2. Feature Extraction

- **Input Frames Are Collected:**
 - A sequence of frames (40 frames) is collected from a video stream using OpenCV.
 - Each frame is resized to 224×224 pixels, which matches the input size expected by **ResNet152**.
- **Preprocessing:**
 - Each frame is preprocessed using the `preprocess_input()` function from TensorFlow's ResNet module.
 - This step scales and normalizes pixel values to be in the same range and format that the ResNet152 model was trained on (ImageNet standards).
- **Passing Through ResNet152:**
 - Each frame (after preprocessing) is passed through the **ResNet152 model**.
 - ResNet152 is a deep convolutional neural network that extracts **high-level features** from the image — like body parts, motion patterns, facial expressions, etc.

- Instead of using the final output layer (which would classify ImageNet categories), we likely **remove the top classification layers** and use intermediate outputs, typically after layers like `AveragePooling2D` and `Flatten`.

```
import os
import sys
import numpy as np
import cv2
import tensorflow as tf
from tensorflow.keras.models import load_model, Model
from tensorflow.keras.applications.resnet import ResNet152, preprocess_input
from tensorflow.keras.layers import AveragePooling2D, Flatten
import time
from collections import deque

print("TensorFlow Version:", tf.__version__)
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU'))))

# --- Configuration ---
# Frame Processing
IMG_HEIGHT = 224
IMG_WIDTH = 224
IMG_CHANNELS = 3
SEQUENCE_LENGTH = 40

# Prediction
PREDICTION_THRESHOLD = 0.2
CLASSES = ['no fight', 'fight']
PREDICTION_INTERVAL = 10 # Predict every N frames after the initial sequence
```

- **Feature Vectors Generated:**

- After frames are processed by ResNet152, they are converted into **feature vectors** (numeric representations of important patterns in each image).
- These vectors capture **spatial features** from each frame: what's happening visually in the image, without explicitly encoding labels.

- **Sequence of Features:**

- Since the model uses `SEQUENCE_LENGTH = 40`, the system gathers **40 such feature vectors** (one per frame).
- This sequence captures the temporal (motion) aspect of the video — helping distinguish between sudden aggressive movements (fight) and calm behavior (no fight).

- **Feeding to a Classifier:**

- These extracted features are then passed to a classification model — LSTM
- The classifier uses these feature vectors to predict whether the sequence represents a violent activity ('fight') or not ('no fight').

9.3. Model Building and Training

```
# --- Model Loading ---
def load_models(rnn_model_path):
    """Loads the ResNet feature extractor and the RNN decoder."""
    print("Loading models...")
    # 1. Encoder (ResNet152 Feature Extractor)
    try:
        base_model = ResNet152(include_top=False,
                                input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS),
                                weights='imagenet')
        for layer in base_model.layers:
            layer.trainable = False
        op = base_model.output
        x_model = AveragePooling2D((7, 7), name='avg_pool')(op)
        x_model = Flatten()(x_model)
        encoder_model = Model(inputs=base_model.input, outputs=x_model, name="ResNet152_Encoder")
        feature_dim = encoder_model.output_shape[1]
        print(f"Encoder Model loaded. Feature dimension: {feature_dim}")
    except Exception as e:
        print(f"Error loading ResNet152 base model: {e}")
        sys.exit(1)
```

• Model Initialization:

- ResNet152 is loaded without its top classification layer (`include_top=False`) and with pre-trained ImageNet weights. This allows the model to serve as a **visual feature extractor** for each video frame.

• Freezing Layers:

- All layers of ResNet152 are set to `trainable = False` to **freeze weights** during training, ensuring the model retains its learned features and speeds up training.

• Feature Pooling and Flattening:

- The final output of the base model is passed through:
 - `AveragePooling2D((7, 7))`: reduces spatial dimensions and retains key features.
 - `Flatten()`: converts the output to a **1D feature vector**.

• Model Wrapping:

- A new `encoder_model` is created using the original ResNet input and the new flattened output, named "ResNet152_Encoder".
- The **feature dimension** of this encoder is printed, which will be used as input to an RNN for violence classification.

9.4. Model Evaluation and Testing

The trained model is evaluated on the testing dataset using the following metrics:

- **Accuracy:** Proportion of correctly classified frames.
- **Precision:** Ability to correctly identify true positive cases (e.g., actual violent activity).
- **Recall:** Ability to detect all relevant instances.
- **F1-Score:** Harmonic mean of precision and recall, balancing both false positives and false negatives.

9.5. Library Used:

- TensorFlow/Keras: Used for building, training, and loading deep learning models (ResNet152 and RNN classifier).
- OpenCV (cv2): Handles video capture, frame extraction, resizing, and real-time visualization.
- NumPy: Supports numerical operations and handling arrays of image features.
- os: Manages file paths and checks for the existence of input/output files.
- sys: Used for exiting the program gracefully in case of errors.
- time: Measures frame processing time and calculates frames per second (FPS)

```
import os
import sys
import numpy as np
import cv2
import tensorflow as tf
from tensorflow.keras.models import load_model, Model
from tensorflow.keras.applications.resnet import ResNet152, preprocess_input
from tensorflow.keras.layers import AveragePooling2D, Flatten
import time
from collections import deque

print("TensorFlow Version:", tf.__version__)
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))

# --- Configuration ---
# Frame Processing
IMG_HEIGHT = 224
IMG_WIDTH = 224
IMG_CHANNELS = 3
SEQUENCE_LENGTH = 40

# Prediction
PREDICTION_THRESHOLD = 0.2
CLASSES = ['no fight', 'fight']
PREDICTION_INTERVAL = 10 # Predict every N frames after the initial sequence
```

10 Result and Evaluation

Smart Surveillance is a video-based monitoring system that detects violent and weaponized activities with high precision, achieving 97% accuracy on static video datasets for faster threat detection and enhanced situational awareness.

10.1 Experimental Setup:

The experimental setup consisted of:

- **Hardware:** Intel i7 Processor, 16GB RAM, NVIDIA RTX 3060 GPU.
- **Software:** Python 3.10, TensorFlow, OpenCV, NumPy, scikit-learn.
- **Datasets:** Real-Life Violence Situations (RLVS)
- **Environment:**

Testing was conducted on static video datasets.

- **Frame Rate:** Videos were processed at 30 FPS after frame extraction and preprocessing

10.2 Testing Strategy:

- **Unit Testing:**
 - Individual modules such as frame extraction, motion detection, feature extraction using ResNet152 sequence classification were independently tested.
- **Functional Testing:**
 - Simulated real-world scenarios using unseen video footage to validate system performance from a user perspective.

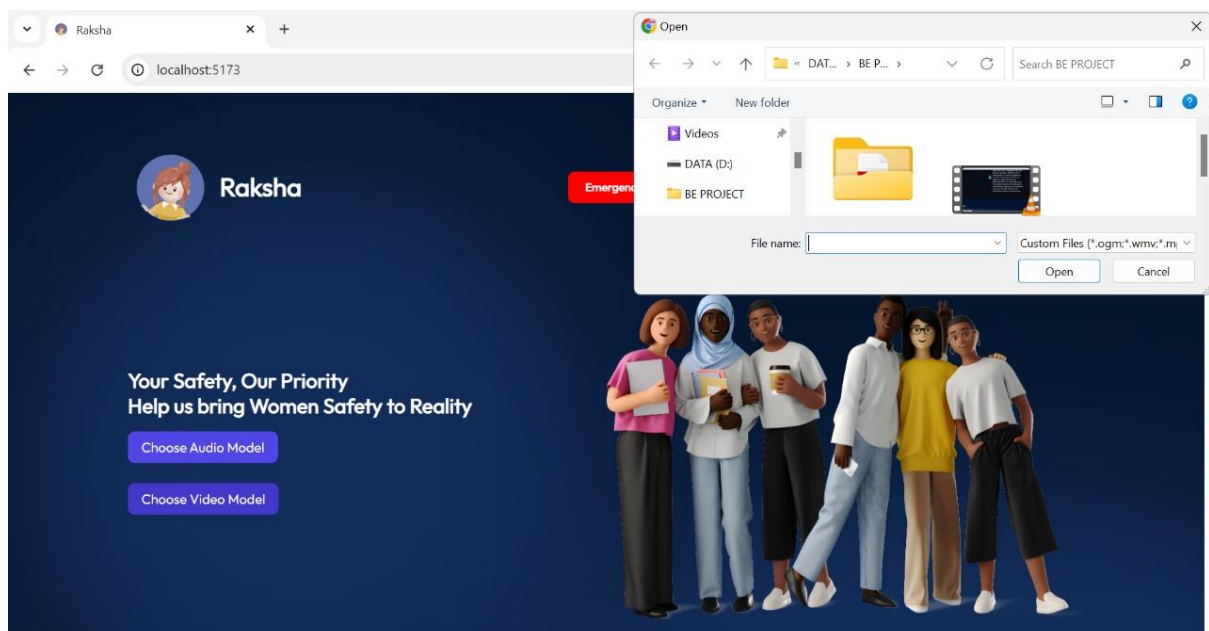
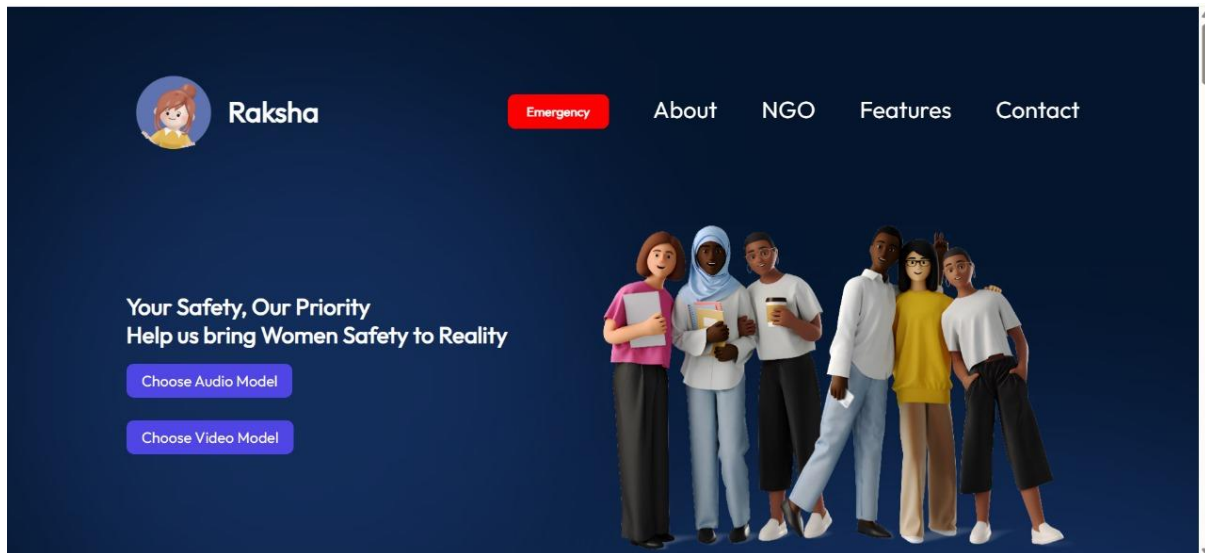
Input Data(Video)



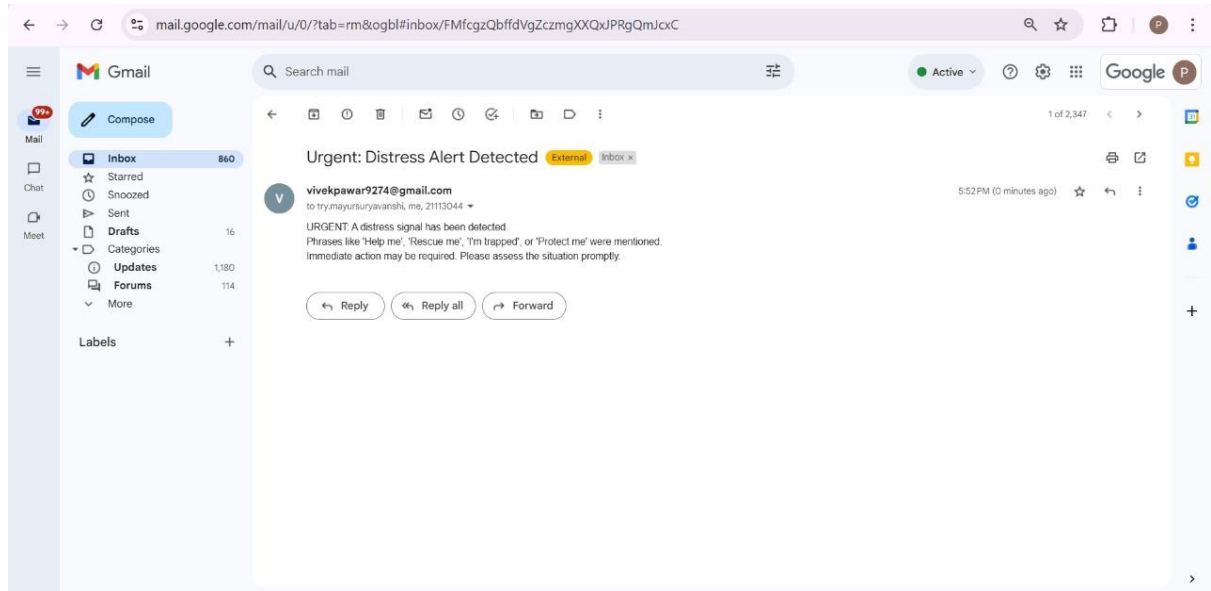
Prediction(Fight Detected)

11 GUI

Login Page



Alerts/Notification



12 PROJECT PLAN

10.1 Project Resources:--

1. People

- Student 1: Model training, dataset preparation
- Student 2: UI/UX design, alert system
- Student 3: System integration and backend
- Student 4: Documentation, testing, project management

2. Hardware

- High-performance processor (for real-time inference)
- High RAM ($\geq 8\text{GB}$)
- GPU for model training
- Storage for video and logs
- Power backup for continuous surveillance

3. Software

- OS: Linux/Windows
- Frameworks: TensorFlow, PyTorch, OpenCV
- Frontend: React, Bootstrap
- APIs: Email SMTP

4. Tools

- IDE: VSCode, PyCharm
- Version Control: Git
- ML Libraries: YOLOv5/YOLOv8, OpenCV, Scikit-learn

10.2 RISK MANAGEMENT W.R.T. NP-HARD ANALYSIS

10.2.1 Risk Identification:-

ID	Risk Description
1	Real-time detection of activities
2	Model integration with frontend
3	False positives/negatives during detection
4	Privacy & legal concerns with surveillance
5	Data storage overload or loss

10.2.2 Risk Analysis

ID	Risk Description	Probability	Impact	Schedule	Quality	Overall
1	Inference bottlenecks	High	High	Medium	High	High
2	Environment-based inaccuracies	Medium	High	Medium	High	High
3	Detection errors	Medium	High	Medium	Medium	Medium
4	Privacy/legal issues	Low	High	Low	High	Medium
5	Storage issues	Medium	Medium	Medium	Medium	Medium

10.3.3 Risk Mitigation

- **ID 1:** Use optimized models, apply pruning/quantization
- **ID 2:** Train with diverse datasets, use adaptive thresholds
- **ID 3:** Employ ensemble methods to reduce misclassifications
- **ID 4:** Use secure data protocols, comply with local surveillance laws
- **ID 5:** Implement auto-archive, log rotation, and cloud storage backups

10.3 TEAM ORGANIZATION :-

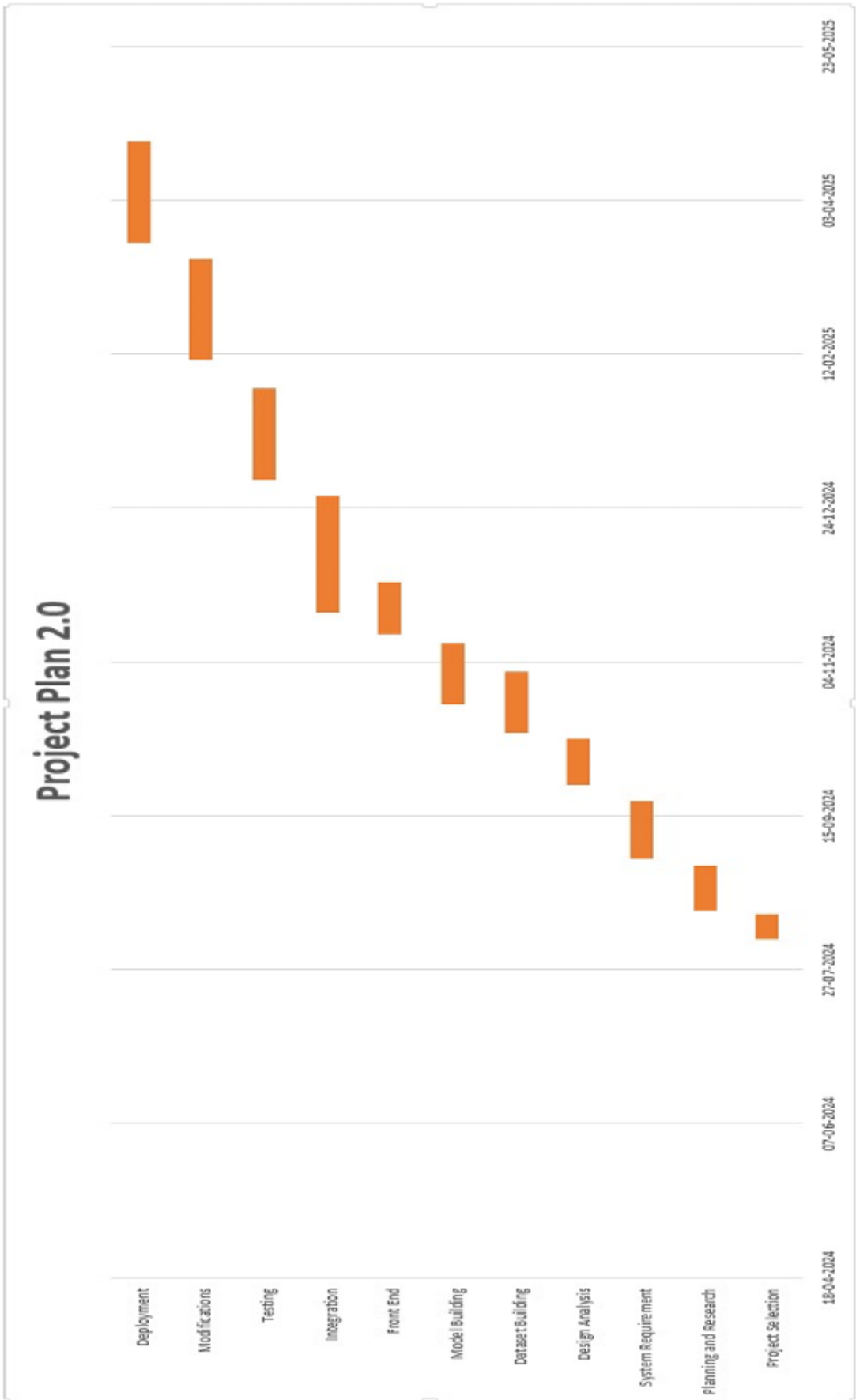
10.4.1.1 Team Structure:-

Role	Responsibility
Team Leader (Student 4)	Supervise project progress, manage tasks, handle documentation
Core Developer (Student 1)	Train models, develop detection modules
Frontend & Alert System (Student 2)	UI dashboard, alert mechanisms, notification integration
Backend & Integration (Student 3)	Backend API, database handling, logging, and module integration

10.4.1.2 Management and Communication:-

- Weekly team syncs (online/offline)
- Use tools like GitHub Projects, Trello, and WhatsApp for task tracking
- Regular updates to project guide via mail and live presentations
- Iterative reviews and sprint-based development cycle

10.5 Project Plan



13 Conclusion

The proposed system successfully demonstrates the potential of AI-powered surveillance technology to enhance public safety through real-time detection of violent activities and incidents of harassment, especially towards women. By integrating camera and microphone inputs with advanced deep learning models for both video and audio analysis, the system is capable of identifying critical situations such as physical fights or distressful screams.

Through modules like frame-based activity classification and acoustic signal detection, the application ensures timely alerts are generated and sent to the concerned authorities, reducing reliance on manual monitoring and response time. The system's ability to function across environments such as malls, streets, and public spaces underscores its scalability and real-world applicability.

Overall, this intelligent surveillance solution presents a significant step forward in leveraging technology for proactive threat detection, ultimately contributing to safer public environments and empowering authorities with actionable insights.

14 References

1. A. Banerjee, L. Wang and H. Kim, "Towards trustworthy human suspicious activity detection in video surveillance," *Artificial Intelligence Review*, vol. 57, no. 2, p. 10630, 2024.
2. J. Chen, Z. Li and Y. Wang, "A survey on deep 3D human pose estimation: Methods and challenges," *Computer Vision and Image Understanding*, vol. 211, p. 103198, 2021.
3. A. Gupta, R. Sharma and P. Verma, "Enhancing motion detection in video surveillance systems," *Journal of Visual Communication and Image Representation*, vol. 84, p. 103237, 2022.
4. M. Kumar, A. Yadav and N. Tiwari, "Suspicious human activity recognition from surveillance video using deep learning," *Electronics*, vol. 11, no. 24, p. 4210, 2022.
5. Y. Luo, Z. Wu and H. Xu, "Abnormal event detection for video surveillance using an enhanced two-stream fusion method," *Expert Systems with Applications*, vol. 207, p. 117662, 2022.
6. R. Shinde, P. Deshmukh and A. Kale, "Suspicious human activity detection using deep learning techniques," *International Journal of Scientific Development and Research (IJS DR)*, vol. 8, no. 3, 2023.
7. P. Singh, S. Patel and M. Mahajan, "Deep learning approach for suspicious activity detection from surveillance video," *International Journal of Computer Applications*, vol. 183, no. 12, pp. 1–8, 2022.
8. T. Wang, X. Zhang and L. Zhao, "Classifying behaviours in videos with recurrent neural networks," *Pattern Recognition Letters*, vol. 154, pp. 56–64, 2021.
9. J. Xu, C. Shen and X. Wei, "Abnormal event detection for video surveillance using an enhanced two-stream fusion method," *Neurocomputing*, vol. 527, pp. 168–179, 2023.

15 Annexure A

Plagiarism Report of Paper



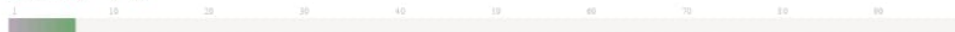
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

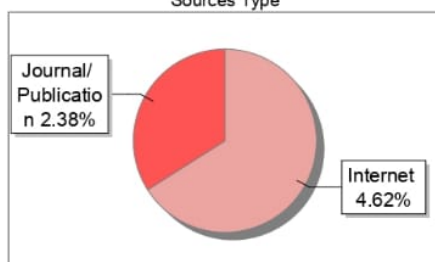
Author Name	Mayur Suryavanshi
Title	Raksha: Enhancing Women's Safety through AI- Based Violence Detection
Paper/Submission ID	3775159
Submitted by	vvg_lib@pvgcoet.ac.in
Submission Date	2025-06-10 17:11:58
Total Pages, Total Words	4, 1722
Document type	Research Paper

Result Information

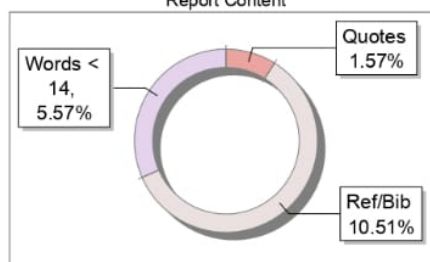
Similarity **7 %**



Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

7

SIMILARITY %

11

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	arxiv.org	1	Publication
2	ejurnal.seminar-id.com	1	Publication
3	www.freepatentsonline.com	1	Internet Data
4	worldwidescience.org	1	Internet Data
5	moam.info	1	Internet Data
6	frontiersin.org	1	Internet Data
7	en.wikipedia.org	1	Internet Data
8	refubium.fu-berlin.de	1	Publication
9	www.ijnrd.org	1	Internet Data
10	www.mdpi.com	1	Internet Data
11	www.thefreelibrary.com	1	Internet Data